

Algorithms in computational biology

4

4.1 Sequence alignment

In the discipline of bioinformatics known as “sequence alignment,” researchers are working to create computer-based methods for comparing and locating amino acid or DNA base pair sequences that are comparable. The most fundamental aspect of manipulating biological sequences is alignment, which finds use in a variety of contexts, including sequence assembly, sequence annotation, structural and functional predictions for genes and proteins, phylogeny, and evolutionary research. Maybe in 1966, the edit distance between two strings was defined as the smallest number of insertions, deletions, and letter swaps needed to change one string into another. This gave rise to the computer challenge of sequence alignment (Saloom et al., 2022).

4.1.1 Local alignment

Local alignment is possibly the simplest technique to compare two sequences; it is the approach that employs the fewest assumptions regarding how the similarity should be structured: one is tasked with locating any subsequences with a similarity greater than a predetermined threshold. Finding evolutionarily constrained elements, all pairs of genes, other repeats, including transposons, as well as any other commonalities is rather easy using this approach. Local alignments seem to be more useful when two different sequences are thought to have parts that are the same or have similar patterns. Local alignments get rid of any parts that don’t match and only align the parts that stay the same between two sequences. This makes sure that the mismatched parts don’t show up in the final result.

4.1.2 Global alignment

It is an attempt to align the whole sequences that are similar to some extent and have approximately the same length, for example, orthologous and paralogous sequences. In it, the complete sequence of protein and nucleotides is compared from beginning to end, and vertical bars present between the amino acids represent the identical residues. The classical technique for global alignment is known as the Needleman-Wunsch

algorithm and is dependent on dynamic programming. The algorithm is used to align strings of proteins and nucleotide sequences. The advantages of global alignment are: It is easy to understand, provides complete sequence pairing in output, can examine the small variations between two sequences, and also helps find polymorphism between two sequences. The limitation of global alignment is that in divergent sequences or strings of different length, this method is not capable of giving an exact result as it fails to identify extremely similar local sites among two sequences (Sun et al., 2018).

4.1.3 Gap penalty

4.1.3.1 Gaps and gap penalties

Aligning DNA or protein sequences allows us to estimate their distances, which is useful for inferring molecular function by discovering similarities with known functions. For this reason, a scoring system is required to determine whether an alignment is a “good alignment” of sequences. Therefore, we may learn what kinds of alignments to examine and assess the alignment’s significance by using alignment scoring. In the scoring system, there are three outcomes: a match, a mismatch, and a gap (Fig. 4.1). Matches are rewarded with positive values or high scores, while mismatches are penalized with negative values or low scores. This seems pretty easy to do when it comes to DNA, but scoring protein sequences is harder because alignment scoring has to take into account the physiochemical properties of amino acid residues.

While the concepts of match and mismatch are simple to comprehend, the notion of adding gaps is complex. A gap in the sequences indicates the deletion or insertion of amino acid residues or nucleotide bases. When inserting a gap, the following issues could come up: How many gaps are allowed? How long can they last? How do you choose where to put them? Supposedly, it can be attempted to enhance the alignment score by inserting several gaps, but would this be biologically significant? It would seem logical that there must be a problem with this strategy. When we

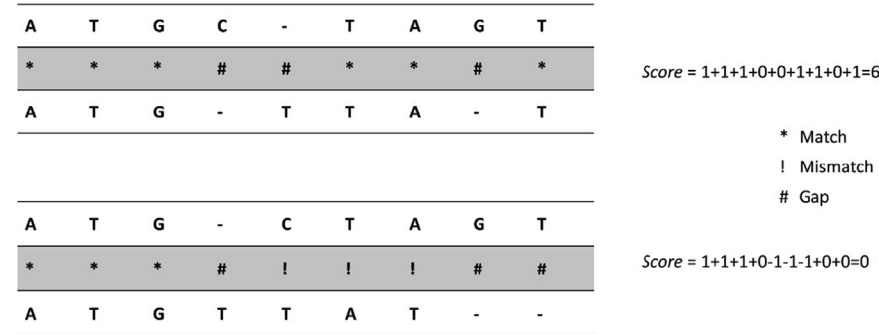


FIG. 4.1
Representation of match, mismatch and gap.

examine dynamically produced sequence alignments, the number of gaps is always restricted. It appears that the algorithms are given instructions to limit the frequency of gaps and their location in the sequence. This directive is referred to as the “gap penalty.” Every time the algorithm inserts a gap, it generates a penalty score, which can either lower or raise the alignment’s overall score. Each time an alignment is performed, the gap penalty can be modified as a separate parameter. The total number of gaps, their length, and their location in the sequence alignment can all be altered by changing the value of the gap penalties. The match, mismatch, and gap penalties are correlated in the following way: The mismatch and match scores can be used to determine the sequence alignment and its statistical significance if the gap penalty is higher than those scores. If a significant penalty is assigned for both gaps and mismatches, then only matches will be evaluated and checked for significance. While the longest common subsequence may be determined if the gap penalty is zero and the match score is much lower than the mismatch, testing the significance of this subsequence is typically challenging (Fig. 4.2).

There are three types of gap penalty:

Constant or fixed gap penalty: This is the most common and basic sort of gap penalty, where a constant negative score is awarded to the gap irrespective of its length. This kind of gap penalty pushes the algorithm to leave bigger continuous areas and create fewer, larger gaps. For instance, a sequence has three gaps and four matches, and the score assigned to all the gaps is -1 rather than being given individually. In this case, the sequence’s overall alignment score will be $4 - 1 = 3$.

Linear gap penalty: There is an individual score for each gap in this part. The length (L) of each indel in the sequence is considered by the linear gap penalty. If the penalty for each indel residue is A and the gap length is L , the total gap penalty will be the product of the two “ AL .” So, using the constant gap penalty as an example, the total alignment score of the sequences will be $4 - 3 = 1$. Because the overall score declines with each new gap, this approach is more favorable for sequences with gaps that are shorter in length. As we increase the gap penalty, the score will immediately increase as well, resulting in alterations to the alignment’s significance.

Affine gap penalty: the most often employed gap penalty that penalizes indels. The affine gap penalty combines the variables of both linear and constant gap

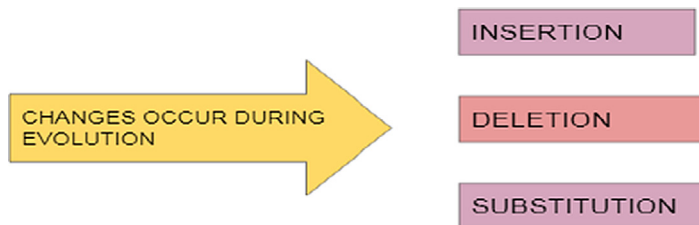


FIG. 4.2

Gap penalty.

penalties, yielding the form $X + AL$, where L represents the gap length and X and A represent gap opening and gap extension penalties, respectively. Opening penalty is assigned when a gap is first introduced in the alignment of the sequence, if this value is increased, the gaps will be less frequent. The “gap extension penalty” is referred to as the penalty for whenever the gap is extended by one or more residues after gap opening. The gaps will become shorter if the extended penalty value is increased. There will be no penalty for terminal gaps.

The values of A and X are not always obvious since they depend on context. If the goal is to locate matches that are closely connected to one another, it is correct approach to employ a greater gap penalty so as to minimize the number of gap openings. On the other side, when the goal is to find a more distanced match, the gap penalty should be decreased. The size of the gap is also impacted by the relationship that exists between X and A (Fig. 4.3).

4.2 Pair-wise alignment

PSA is a technique to align two sequences that searches for the best and most efficient pairwise alignments of a few query sequences using a database similarity search tool. The method has found widespread application in the study of sequences for their functional, evolutionary, and structural properties. When matched sequences reveal a high degree of similarity, the two sequences can be considered members of the same family. Pairwise alignments are used to compare only two sequences at once. They are easy to calculate and are usually used for tasks that don't require a high level of accuracy.

In order to align anything less than an exact alphabetic match, the algorithm must be aware of what it is looking for and how to evaluate the significance of what it finds. In order to do this, “comparison matrices” have been developed, defining a value for each and every potential match scenario—effectively a score of how

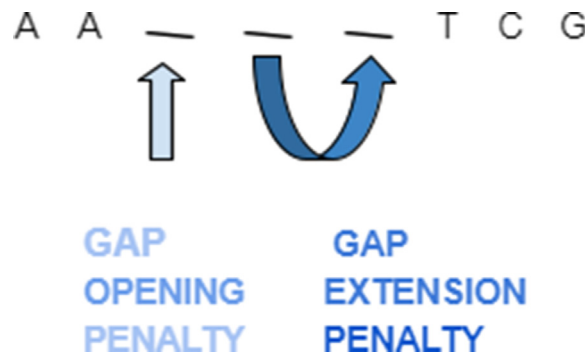


FIG. 4.3

Description of GAP opening penalty and gap extension penalty.

well the computational alignment is performing. The algorithm will look for the best possible score. The total score can only be used for the alignment it creates. It can't be used for anything else.

The aim of pairwise sequence alignment is to find the best pairing of two sequences. K-tuple method, dot-matrix technique, and dynamic programming are the three most used approaches for pairwise alignments. All three approaches have pros and cons, but they all have trouble matching highly repeated sequences with little relevant information, especially when the number of repetitions in the two sequences to be aligned is different.

4.3 Dot-matrix method

A dot matrix approach compares two sequences for potential alignment. In this simplistic approach, the two sequences to be matched are laid out as axes on a grid, and a dot is added at each position where they are an exact match. The top of the matrix lists the first sequence, indicated by the letter X, while the left side lists the second sequence, indicated by the letter Y. Beginning with the first character in Y, one proceeds across the column while remaining in the first row and adding a dot in each column when the character in X is the same. The procedure is repeated once all potential comparisons between X and Y have been performed. The sections of a matching sequence will then be graphically represented by diagonal lines. Any region of similarity may be identified with the use of a diagonal line of dot markers. Dots that aren't on the diagonal but are otherwise isolated signify random matches.

The dot-matrix technique, which generates a group of alignments for given individual sequence sections, is qualitatively and theoretically simple, but evaluating it on a large scale takes a long time. The dot matrix method is computationally available as "Emboss DOTMATCHER," where dot plots can be made as easily as possible. A dot-matrix plot can be used to visually identify sequence properties like insertions, deletions, repetitions, and inverted repeats in the absence of noise. Some implementations change the size or intensity of the dot depending on the degree of similarity between the two characters to accommodate conservative substitutions. The dot plots of closely related sequences will be combined into a single line along the matrix's primary diagonal. Filtering out random matches with a sliding window can make it easier to locate regions that match. It means that more than one nucleotide or amino acid position can be aligned at the same time, and that a dot will only be given if a certain number of matches can be made.

Dot plots have drawbacks as a method of displaying information, including noise, a lack of clarity, unintuitiveness, and difficulties in obtaining information on match positions and summary statistics for the two sequences. There is a lot of unused space in dot-plots since they can only display two sequences, and because the match data is automatically reproduced across the diagonal, noise or empty space takes up a large portion of the plot's real size. By comparing each character using a threshold value and window size, the noise problem may be resolved (the

size can be assigned according to the requirement). For a dot to be drawn, a certain number of the matches in the window must be right.

The amount of repetition in a sequence may also be assessed using dot plots. When plotted against themselves, parts of a sequence with a lot in common appear as lines off the main diagonal. When a protein contains several similar structural domains, this phenomenon might occur.

The interpretation of dot matrices is as follows: areas of resemblance will show as diagonal runs of dots; inversions will be indicated by reverse diagonals that are perpendicular to the diagonal; and palindromes will be indicated by reverse diagonals that cross the diagonal (Fig. 4.4).

Dot plots provide several benefits, including the fact that they are quite simple to implement. Its presentation makes it simple to comprehend. It illustrates every combination of aligned pairs that is feasible. It is possible to employ it in conjunction with other different approaches; it finds inverted and direct repeats, insertions, and deletions much easier than the other, more automated approaches do. One drawback shared by most dot matrix computer programmes is that they do not display an actual alignment. There is no score that is returned to show how ‘optimal’ a certain alignment is (there is no statistical significance that could be examined) (Rice et al., 2000).

4.4 Dynamic programming

Dynamic programming is a technique for segmenting longer sequences into manageable chunks, with each transition between pairs of characters in an alignment accounting for all potential modifications.

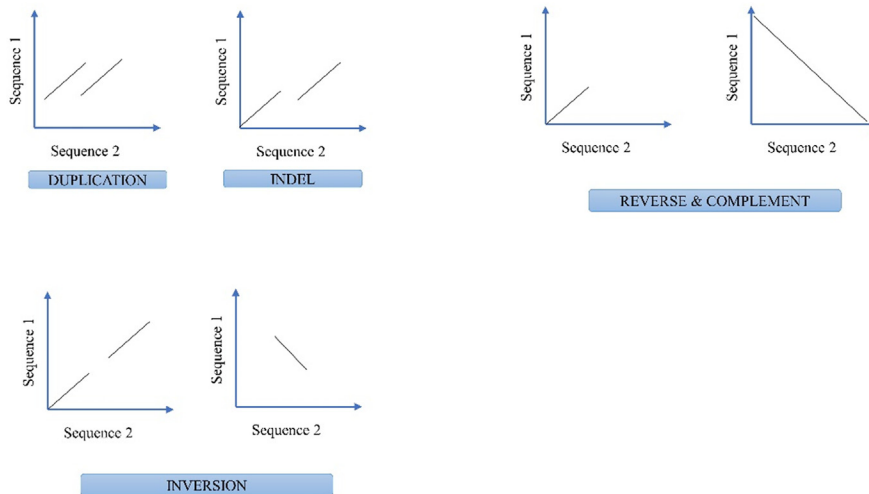


FIG. 4.4

The patterns for interpretation of dot plot results.

In the field of computational biotechnology or bioinformatics, dynamic programming has emerged as the dominant paradigm. Common uses of this information include protein-DNA binding, protein folding, gene recognition, RNA structure prediction, and sequence alignment. The exon/intron structure of eukaryotic genes is determined using DP, which is also applied for assembling DNA sequence information from the segments provided by automated sequencing.

Dynamic programming is based on the idea that solutions to smaller problems can be saved and used later in the larger problem. Comparing this solution with the next solution in the series is normal protocol; the ideal solution will then be compared with the next solution, and so on, till the entire sequence has been exhausted. An algorithm for dynamic programming consists of four components: a recursive formulation of the optimum score; a DP matrix for storing subproblem optimal scores, a method of filling the matrix from the bottom up by tackling the most elementary subproblems first; and a method of tracing back through the matrix to identify the details of the best solution that produced the highest score (Naghizadeh et al., 2021).

The process of matching nucleotides to protein sequences, which is complicated by the need to take care of frameshift mutations, can be aided by dynamic programming. The method is particularly helpful for sequences with many indels since it may detect frameshifts offset by any number of nucleotides, which makes it challenging to align using more effective heuristic techniques. In practice, the approach requires a system built expressly for dynamic programming or a lot of computing power. The programmes BLAST and EMBOSS offer fundamental tools for producing translated alignments. More broad techniques are offered by the open-source application GeneWise. Given a certain function of scoring, the dynamic programming approach is meant to produce the best alignment; however, selecting a suitable function of scoring is often an empirical rather than a theoretical procedure. Dynamic programming is slow when it has to deal with a lot of sequences or sequences that are too long, even though it can include more than one sequence.

Two classical algorithms of dynamic programming are:

Needleman and Wunsch (1970)—For global alignment and the result contains all residues in the alignment (Needleman and Wunsch, 1970).

Smith and Waterman (1981)—For local alignment and the result contains only certain parts of our sequences (Smith and Waterman, 1981).

4.4.1 Needleman-Wunsch

A bioinformatics technique called the Needleman-Wunsch approach is used for the purpose of aligning protein or nucleotide sequences. When comparing biological sequences, this was one of the first instances of “dynamic programming.” The method was developed by Saul B. Needleman and Christian D. Wunsch, and it was first presented in the year 1970. The approach divides a significant challenge, such as the whole of the sequence, into a series of more manageable issues, and then makes use of the solutions found for the more manageable issues to locate an optimal

response to the significant challenge. It is also known as the optimum matching algorithm and the global alignment approach. Even today, the Needleman-Wunsch method is frequently utilized, particularly when the quality of the global alignment must be maintained. The technique gives a score to every possible alignment, and its goal is to identify all of the alignments that have the greatest significant value.

The first component of Needleman and Wunsch's ultra-algorithm generates all possible alignments between any pair of sequences, considering their probabilities of being similar, distinct, or containing some insertions along with deletions. When the first phase is complete, a scoring mechanism is implemented to assign a value to each base pair or nucleic acid combination. For example, if 2 Gs are aligned, they receive a score of 1 because they are a perfect match; if C and T are aligned, they receive a value of -1 because they are incompatible; however, if there is no base pair or amino acid available, it is termed a "gap penalty" and receives a score of 0.

Following the completion of all of these steps, the total scores need to be summed in order to select the best alignment possible from among all of the potential alignments that were generated by the procedure. It is necessary to select the alignment with the highest possible score.

Creating a two-dimensional (2D) matrix using the penalty scores for the match, mismatch, and gap is a necessary step in the process. The matrix is solved in three stages: the first stage is called initialization, the second stage is called matrix filling, and the third stage is called traceback (Fig. 4.6).

Seq1: ATTAC

Seq2: AATTC

4.4.1.1 Step 1: Initialization table "T"

The first step in the algorithm is the initialization, in which a scoring matrix is made. The formation of a scoring matrix begins with the placement of sequences, which are placed on the x and y axes of the matrix.

While Seq1 "ATTAC" will be positioned at the "x" coordinate, Seq2 "AATTC" will be positioned at the "y" coordinate. The first column and row of the matrix are initially started from the (0,0) cell, with 0 being the first value in the cell (Fig. 4.6). The gap score is added to the adjacent cell of rows and columns.

4.4.1.2 Step 2: Filling the matrix

The second and arguably most important stage of the process is filling up the matrix, beginning in the top-left corner. It is necessary to know the scores of the cells on the diagonal, left, and right in order to get the maximum score of each cell. Match or mismatch (assumed) scores are added to the diagonal score. In a similar fashion, the gap score is added to the values coming from adjacent cells or boxes (horizontal and vertical). With these three numbers at your disposal, the highest possible score can be obtained; use that to fill the *i*th and *j*th slots.

If a value comes from the diagonal direction or cell, it is determined if the residues are identical or not. They are added based on their alignment match or

mismatch score. As an outcome, three values come from three independent directions. However, since maximum match alignment is required, only the highest value is placed in the cell (Fig. 4.5)

- Scoring method:
 1. +1 score for every match found.
 2. -1 score for every mismatch.
 3. -1 gap penalty for every insertion and deletion.

4.4.1.3 Step 3: Traceback

After the matrix has been filled in, the final score to be computed is the score of the best possible alignment of the whole sequences. Nevertheless, the best alignment has yet to be determined. This is found by a recursive matrix “traceback.” The last and most important step in the sequence alignment process. Starting with the bottom right corner cell, the algorithm determines which of the three highest values was used to fill this cell, and the direction from which that value came is highlighted or saved with a back arrow, before moving to that cell to find the best path or alignment. To fully construct the optimum alignment, the procedure is repeated until the cell (0,0) is reached.

Recursion:
$$D_{i,j} = \max \begin{cases} D_{i-1,j-1} + s(a_i, b_j) \\ D_{i-1,j} + s(a_i, -) \\ D_{i,j-1} + s(-, b_j) \end{cases} = \max \begin{cases} D_{i-1,j-1} + 1 & a_i = b_j \\ D_{i-1,j-1} + -1 & a_i \neq b_j \\ D_{i-1,j} + -2 & b_j = - \\ D_{i,j-1} + -2 & a_i = - \end{cases}$$

FIG. 4.5
Function for matrix filling.

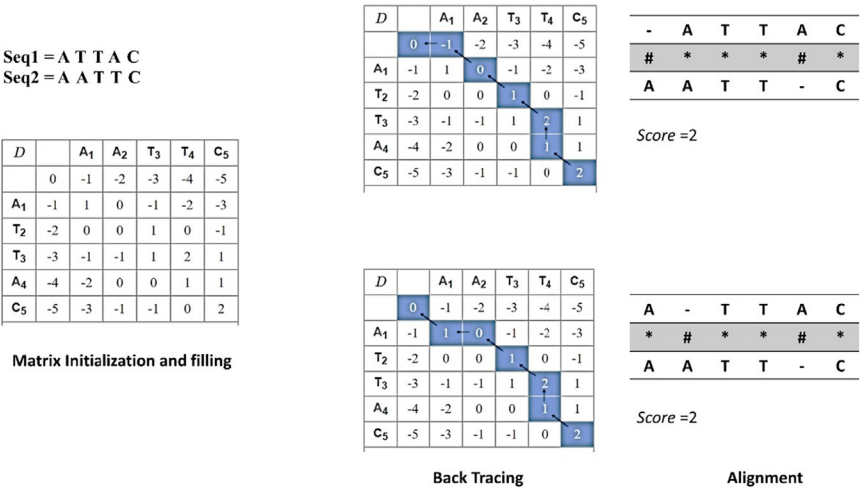


FIG. 4.6
Needleman Wunch algorithm steps.

4.4.2 Smith Waterman algorithm

Temple F. Smith and Michael S. Waterman were the ones who published the strategy for the first time in 1981. The global alignments are produced via the Smith-Waterman method. Protein alignments commonly employ a substitution matrix to provide points for amino acid matches and mismatches, as well as a gap penalty for connecting an amino acid from one sequence to a gap in another. Although in practice a positive match score, a negative mismatch score, and a negative gap penalty are more prevalent, a scoring matrix can be utilized in the alignments of DNA and RNA. However, such results may be explained by altering the procedure. The application of two distinct gap penalties for opening and extending a gap is a frequent expansion of standard linear gap expenditures. The former is typically far higher than the latter; for example, -10 for a gap that is open and -2 for a gap that is extended. As a result, gaps between residues and gaps in alignments are often kept to a minimum, which makes biological sense.

Smith-Waterman's algorithm deals with a segment of the total gene. Rules for filling the scoring matrix in local alignment are the same as global alignment, except that instead of putting values in scoring matrices in real values like -3 , -1 , -5 , $+2$, $+4$, the program puts values only in binary numbers, i.e., 1 or 0. Any negative value in the scoring matrix will be represented by a 0, and any positive value will be represented by a 1.

The sequences were given the following scores: match = $+1$, mismatch = -1 , and gap = -1 . All the negative resultant values will be represented as zero, such as in the cells of T1 and G1. However, all the positive values remain the same as calculated. The next step is trace backing, starting from the highest value and tracing backing until the value of zero is not obtained in the first row or column. In the given example, the highest value is "2" in three different cases, the first being C3 and C2, the next being C3 and T3, and the last being C3 and A4. Three different alignments can be seen, and the optimal one can be chosen (Fig. 4.7).

4.4.2.1 Limitations

The exponential growth of genetic data puts the many approaches to DNA sequence alignment that are now in use to the test. In order to fulfill the essential criteria for a method that is both rapid and trustworthy, real-time parallel processing must be accomplished using innovative methods. It has been suggested that optical computing approaches might serve as potential replacements for the electrical implementations that are now in use. One of these methods, known as OptCAM, has been shown to be much more efficient than the Smith-Waterman algorithm.

It was assumed that local alignment shows better alignment when compared to global alignment as they produce patterns that are conserved in whole DNA and amino acid sequences. Researchers say that local alignment can be used in place of the Needleman-Wunsch algorithm for making the different DNA and amino acid sequences, which could consist of a small, matched region or when they are of different lengths, when they are overlapping, or when one sequence is a subsequence of another (Mullan, 2006).

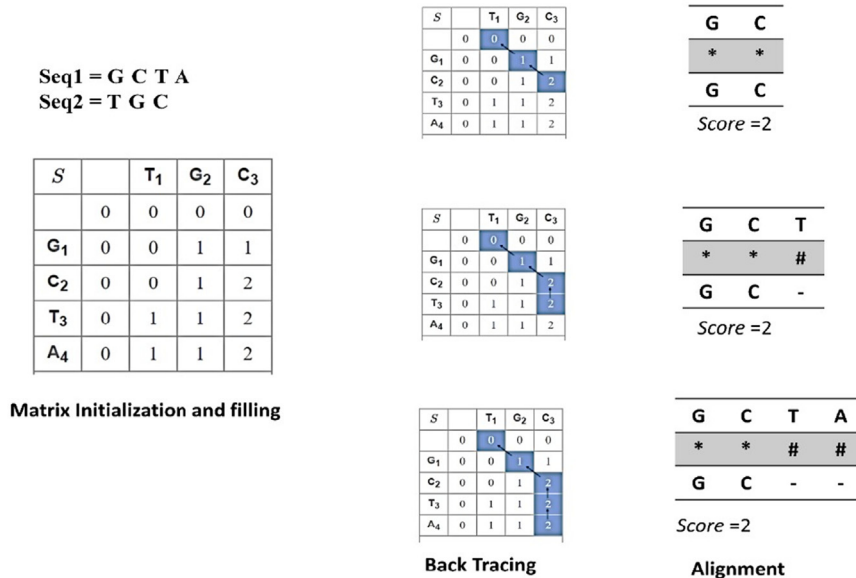


FIG. 4.7

Smith Waterman algorithm.

4.5 Scoring matrices

In dynamic programming, the alignment uses scoring methods to quantify the probability of one residue being replaced by another in alignment; this method is known as the substitution matrix. The scoring method for nucleic acid sequence is relatively simple. A high or positive value is given for a match, and a negative value is assigned to a mismatch. Because amino acids with the same properties are more likely to match than those with similar properties, scoring for amino acids is quite complex due to the influence of physicochemical properties. Substitution of an amino acid with one having different physicochemical properties is disruptive to the role and structure of a peptide. For instance, the aromatic amino acids like tryptophan, tyrosine, and phenylalanine can easily substitute for each other without any disturbance because they all have the same chemical properties. The same happens with basic amino acids like lysine, arginine, and histidine. So, these are more frequent in substitution. But the amino acid cysteine, which can form a disulfide bond, is interrupted by enzymes and can destabilize protein structure; therefore, this kind of amino acid substitution does not occur frequently (Pearson, 2013).

4.5.1 Scoring matrices for amino acids

Substitution matrices for amino acids are 20*20. There are basically two kinds of matrices: first, interchangeable amino acid residues that are feature-dependent.

The second is dependent on empirical studies of amino acids. The first method is less precise than the second approach. The empirical method is the most favored and accepted method, and it consists of PAM and BLOSUM. These are thoroughly based on the statistics of substitution probability, and therefore, by analyzing the likelihood, they obtain a score for the particular alignment. If the one with high substitution scores higher, one of three things can happen: if the score is positive, it means that the frequency of residue substitution in homologous sequences is higher than when it happens randomly. If the score is 0, it represents the frequency that has occurred by chance as being equal to the frequency of substitution in homologous sequence data. If the score is negative, it means the frequency of substituting homologous sequence sets is less than the frequency expected by chance.

4.5.2 PAM (point accepted mutation)

PAM was first created by Margaret Dayhoff and his colleagues in the 1970s to develop substitution matrices of amino acids from related proteins having a minimum of 85% sequence identity. It was used for recreating ancestral groups of 34 superfamilies based on 71 pairs of sequences that were approximately 85% identical and based on 1572 variations. One unit of PAM is defined as 1% of the amino acid residue position that has changed, or one amino acid substitution per 100 amino acids. The family of PAM matrices includes PAM80, PAM120, and PAM250. The number indicates the basis of the matrix, which is the evolutionary distance between the pair of sequences. Because they have a greater evolutionary distance, the greater the digit, the greater the distance. The number of residue replacements used to calculate the PAM1 matrix To deduce the PAM-1 substitution matrix, a class of closely related strings with mutation frequencies corresponding to PAM-1 is selected. A phylogenetic tree can be constructed using the parsimony rule. When pam matrices are compiled, each mutation at the site is independent of any previous change at that site. PAM matrices are symmetrical because the direction of any change or mutation cannot be determined at a given region. It is derived from the global alignment method. It is a linear-space scoring method.

4.5.2.1 PAM score

1. It is a multi-step process that divides the mutational variation that occurred from the common ancestor for any residue by the total residue of amino acids in an alignment.
2. Normalizing expected frequencies of residues through random chance.
3. Known as the “log odd score” on a log scale.

4.5.2.2 Example

Assume C and D are two residues that randomly align; the probability that they aligned together randomly is $\Pr(C,D)$. The likelihood that C and D will meet is related to sequence alignment (orthologs: $Po(C,D)$).

Score function = $\log \text{Po}(C,D)/\text{Pr}(C,D)$.

Scoring matrix = $S(c,d) = \log_{10}(P_{cd}/P_c P_d)$.

By multiplying PAM 1 by itself, you can get other PAMs. For example, you can get PAM 80 by multiplying PAM 1 by itself 80 times, since PAM is defined as one mutation per 100 amino acid residues.

Limitation of PAM:

- The proteins studied had only a small amount of variation (85% identity), and alignment for more divergent sequences was not possible.
- Biased matrix due to its reliance on globular proteins.
- It is only based on one original dataset.

4.5.3 BLOcks Substitution matrix (BLOSUM)

This method was given by Henikoff and Henikoff in 1992 ([Henikoff and Henikoff, 1992](#)), and it overcomes the limitation of PAM matrices. It was developed by a group of around 2000 aligned, ungapped sites from 500 families of proteins and is known as the “BLOCKS” database. It is basically an ungapped alignment of highly conserved sites of protein. The presence of each residue pair in each column of all blocks is counted. Despite the extrapolation function, the actual percentage identity value of the sequence is used to construct the matrix in this case. It represents the sequence alignment with no more than 62% identity. It means that in a number system, BLOSUM is the opposite of PAM, and the lower the BLOSUM number, the more different or far apart a sequence is.

4.5.3.1 BLOSUM62

It represents the sequence alignment with no more than 62% identity. In the case of a numerical system, it means the opposite of PAM; the more divergent or distantly related a sequence is, the lower the BLOSUM number.

4.5.3.2 BLOSUM score

It is the log ratio of the substitution frequency of the observed amino acid residue to the expected probability of a specific residue. The log is taken to base 2, the final value is rounded off to the nearest integer, and then it is written in the substitution matrix.

Comparison between BLOSUM and PAM

More distant sequence

PAM 100 = BLOSUM90

PAM120 = BLOSUM80

PAM160 = BLOSUM60

PAM200 = BLOSUM52

PAM250 = BLOSUM45

4.6 Word methods

Though far more effective than dynamic programming, this method—also known as *k*-tuple methods—does not ensure that the optimum alignment solution will be discovered. These strategies are particularly helpful in large-scale database searches where a significant portion of candidate sequences are projected to share little in common with the intended set of query sequences.

The query is divided into a cassette of short, non-overlapping subsequences (“words”), which are then compared against database sequences. The offset is derived by subtracting the respective positions of the words in the two sequences being compared; if several different words provide the same offset, this indicates a region of alignment. These methods only use more sensitive alignment criteria if this area is found; as a result, many useless comparisons with sequences with no significant similarity are skipped. The FASTA and BLAST families of database search tools are well known for their use of word methods (Altschul et al., 1990).

Software for sequence alignment called FASTA is pronounced “fast A.” David J. Lipman and William R. Pearson initially presented it in 1985. The software package’s initial iteration was only intended to compare protein sequences. Programs for protein:protein, DNA:DNA, protein:translated DNA (including frameshifts), sorted or unordered peptide searches, among other capabilities, are included in the most recent edition. For figuring out whether or not something is statistically significant, the software has a more advanced way of shuffling.

In the FASTA technique, the user specifies a value *k* as the word length to search the database with. The technique is slower but sensitive for smaller values of *k*, making it appropriate for searches with a short query sequence.

The first method to employ a look-up table and the seed searching strategy for indexing the initial sequence is FASTA. The look-up table is used to find perfect match seeds of length “*k*” in both sequences at the start of the process. To locate all seeds across a diagonal between the two sequences, FASTA employs a “diagonal” approach. Different scores are given to seeds depending upon their position; seeds receive a positive score, and intermediate areas receive a negative score. So, clusters of high-scoring seeds add more to the score of local diagonals than clusters of low-scoring seeds. There may be “*n*” diagonal seeds produced throughout the procedure, of which FASTA stores the top 10. FASTA determines the maximum score “init1” as well as “good” diagonals. A diagonal with a score higher than the threshold is considered to be good. Then, by building a directed weighted graph around the seeds, all of these diagonals are integrated into a single, space-allowing, high scoring alignment. The optimum alignment is then identified and designated as “initn” on the most heavily weighted graph. The following phase of FASTA constructs a narrow band cantered along init1 with a width of “*k*.” Next, FASTA uses the Smith-Waterman method to determine a local alignment that is best within this band.

The software primarily saves word-to-word matches of length *k* using a hash table pattern search. The software searches the returned word hits for segments

containing a cluster of neighboring word hits. When the database is searched for all possible sequences, the software plots the scores of each sequence in a histogram and determines their statistical significance. The “E-value” quantifies the possibility that the alignment was caused by random chance alone. The value must be significantly less than 0.05. Very low E values in the results indicate that the sequences are homologs. A long list of steadily decreasing (E) values indicates a large sequence (gene, protein, or RNA) family. Long areas with intermediate similarity are more significant than small sections with high similarity.

FASTA has the drawback of only finding the region surrounding `init1` while excluding the area that contributed to `initn` if the sequences in evaluation include several homology regions (two optimum diagonals). A seed, which is smaller than `k`, could be overlooked. The primary benefit above the optimum algorithm is its speed.

BLAST is one of the NCBI’s tools. BLAST was designed to be a faster alternative to FASTA without sacrificing much accuracy. It uses a `k`-word search like FASTA, but instead of assessing every word match, it just evaluates the most significant ones. The default word length in most BLAST implementations is optimized for the query and database type, and it is only changed in exceptional instances, such as when searching with repetitive or very short query sequences. If the word length is approximately 3 amino acids or 11 bases, it would not require identical words, and if the words are similar, then there will be no alignment. BLAST makes similarity searches very quickly but also makes errors, e.g., it misses some important similarities and makes many incorrect matches. Two websites to discover implementations are EMBL FASTA and NCBI BLAST.

4.7 Multiple sequence alignment

Sequence alignment is the alignment of two or more sequences. The alignment of the sequences is the arrangement of two sequences in relation to one another. Alignment is the process of placing two sequences one after the other and looking for similarity regions and homology between them (Edgar and Batzoglou, 2006). For example, if there are two sequences that need to be aligned, one nucleic acid sequence (like ATCG) and another sequence (like TCA).

From Table 4.1, it can be deduced that there are three ways to align two sequences. We attempt to obtain the optimal alignment, which entails aligning two sequences in a manner that generates the greatest number of matches. Compare the homology or similarity between sequence 1 and sequence 2 in Table 4.2. Alignment is the process of obtaining matches. If there are more matches, then the sequences are more similar, and if there are fewer matches, then the sequences are less similar. Therefore, all feasible alignments of these two sequences are shown below.

There are no matches in Table 4.2, but there are three mismatches with one gap. We are searching for three rules, namely match, mismatch, and gap, to identify sequences. Since this sequence is short, there are fewer alignment options. However,

Table 4.1 First pair of three sequences.

A	T	C	G
	T	C	A
Gap	Match	Match	Mismatch

Table 4.2 Second pair of the three sequences.

Sequence 1	A	T	C	G
Sequence 2	T	C	A	
	Mismatch	Mismatch	Mismatch	Gap

there are still many ways to align two sequences, such as when the software aligns 500–600 bp of nucleotides to align two sequences.

Consequently, there are two matches in [Table 4.1](#) and one match in [Table 4.3](#), but none in [Table 4.2](#). As [Table 4.1](#) received the highest number of matches, this indicates that it is the optimal alignment. When we upload data into a computer, it integrates all of the data and performs every conceivable alignment, looks for the marks distribution system, analyses the outcome, and then provides marks to each alignment. It then displays the alignment with the highest marks. When there is at least some resemblance between two sequences, they can be aligned. It will be harder to match two sequences from, say, plants and vertebrates. Therefore, the closely related sequences must be aligned.

Using multiple sequence alignment (MSA) is a crucial first step in any bioinformatics analysis of protein or nucleotide sequences since it allows researchers to determine how similar their many sample sequences are to one another. It is difficult to determine the evolutionary connection without using MSA, which is hence required for phylogenetic analysis. A phylogenetic tree cannot be constructed without it. In addition to this, it is also used for finding conserved domains ([Gotoh, 1999](#)).

4.7.1 Progressive alignment

In comparison to pairwise alignments, multiple sequence alignments are notoriously difficult to calculate. This method builds a hierarchical cluster of the sequences by

Table 4.3 Third pair of three sequences.

Sequence 1	A	T	C	G
Sequence 2	T		C	A
	Mismatch	Gap	Match	Mismatch

iteratively generating pairwise alignments. The sequence is first aligned using dynamics programming in progressive alignment. It builds an MSA starting with the most related sequences and gradually adds less related sequences or groups of sequences using the dynamic programming method (Taylor, 1986).

The problem with the progressive alignment method is its dependence on pairwise sequence alignment at the initial step for ultimate MSA results. Also, as this method is based on global alignment, it is not compatible with sequence alignments of different lengths. CLUSTAL W is perhaps the most suitable example of progressive alignment, along with MAFFT, T-COFFEE, and MUSCLE (Fig. 4.8).

4.7.2 Iterative method

Similar in operation to progressive approaches, iterative methods repeatedly realign the original sequences and add new sequences to the developing MSA. Since progressive techniques are always included in the final product, and once a sequence is aligned into the MSA, its alignment is not reevaluated, they are dependent on a high-quality initial alignment. While increasing efficiency, this approximation compromises accuracy. To optimize a generic objective function like producing a high-quality alignment score, iterative algorithms can, however, refer back to previously calculated pairwise alignments or subMSAs (Fig. 4.9).

The two software packages PRRN and PRRP are examples of iterative methods. They utilize a hill-climbing method to maximize its MSA alignment score and rectify alignment weights and “gappy” sections of the MSA repeatedly. PRRP performs better when revising an alignment created by a faster approach. MUSCLE (multiple sequence alignment by log-expectation), which is another well-known

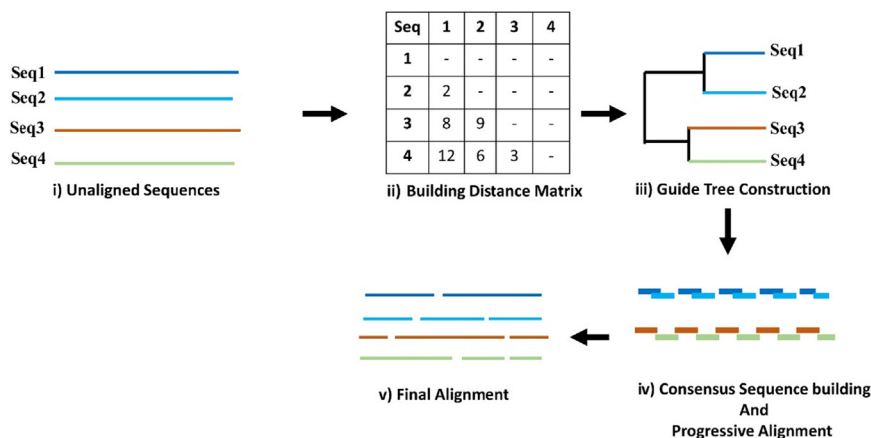
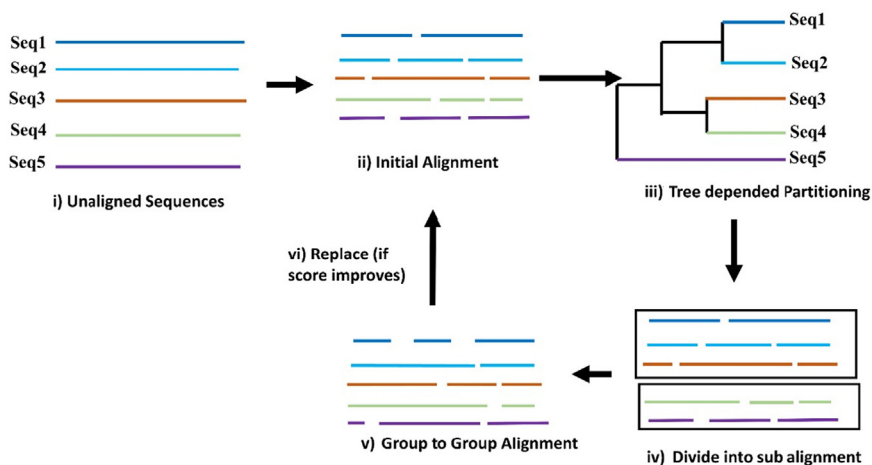


FIG. 4.8

Progressive alignment algorithm.

**FIG. 4.9**

Iterative alignment algorithm.

iterative method, improves progressive methods by using a more accurate distance metric to figure out if two sequences are linked.

4.7.3 MSA filtering

The MSA approaches have several flaws because they rely on heuristic searches that are guided by faulty objective roles. As a result, even if their output is excellent overall, it is invariably polluted with faults. A number of software tools have been created with the goal of filtering MSAs so that only the most reliable parts remain. This is accomplished by removing sites, sequences, or remnants (by replacing them with the gap symbol, “—,” or an ambiguity symbol, “?,” “N,” or “X,” respectively).

It’s crucial to make certain that filtering doesn’t eliminate both the signal and the noise created by the improperly aligned sections. The equilibrium present between noise cutting and signal disappearance is critical in the case of MSA filtering.

Filtering techniques are mainly separated into two divisions. The very first one consists of ways for filtering MSA by completely deleting certain regions or sequences. These methods simply give the user two options for each region and sequence: “take it or leave it,” which is why they are known as TILI-filtering techniques. MSA filtering techniques that function by hiding remnants (replacing them with the gap symbol “—” or a character depicting ambivalence such as “?,” “N,” or “X,” as per the type of sequence) fall into the second group. These are termed “picky filtering” techniques because they grab bits of information from a region or sequence while ignoring the rest.

4.7.4 Filtering techniques' fundamental principles

A. Gaps show locations that are difficult to align and maybe saturated

Sequence alignment involves filling gaps. The number of gaps at a place directly affects how much effort the alignment approach must make; hence, the procedure is more likely to cause mistakes. Biologically, insertions and deletions are less prevalent in proteins than point substitutions. Several gaps show an aberrant evolutionary pattern, perhaps owing to an MSA problem. Multiple mutations at the same location are likely, obscuring the evolutionary signal.

B. Per site, a small number of residues with similar properties are expected

Homologous regions are likely to share traits, notably amino acid sequences. If all amino acids at one site are the same, they all descend from the same hereditary amino acid. Second, the protein has remained functional despite the physicochemical properties of the remnant at this point, and at least 19 substitutions have occurred to do this (which can suggest a site that is saturated). In this situation, removing this alignment piece would be safer. Hydrophobic or positively charged residues should be retained. Estimating residual transformation in one spot may be done in a variety of ways, from easy (a simple count of amino acids found on the site) to complicated.

C. Similarity in homologous sequences is expected

In most processes, homologous sequences are identified by their similarity. This ensures that the sequences that are supposed to be aligned have a minimum value of total similarity. Despite this, it's unusual for a portion of one or more sequences to deviate dramatically from the rest of the alignment. This sequence is unlikely to be homologous to the remaining area, maybe because it was not properly aligned and the segment is homologous to a different portion of the alignment or because it has no homology at all. This latter circumstance can be recognized before sequences are aligned, and various approaches have been developed. This filtering should be done before sequence alignment, since long insertions in particular sequences might hamper the MSA.

D. Orthologous sequences should be consistent across loci (post-filtering)

Alignment filtering algorithms will almost certainly fail to detect a non-orthologous sequence that is homologous with the remaining sequences. However, in the case of a phylogenomic setting, it is conceivable to inspect the MSAs for every locus under consideration. With a high count regarding taxa and genes, it is easier to learn loci, along with the speed of evolution of taxonomic categories. The OrthoMaM v10 process has one basic technique for detecting those sequences that are non-orthologous, whereas Phylo-MCOA offers a more detailed solution. However, this is not applicable in cases where the study is focused on the evolution of an entire family of genes (both paralogous and orthologous).

4.7.5 Programs and methods for multiple sequence alignment

4.7.5.1 *Clustal family*

MSA's most well-known program is the Clustal family. There are two programmes in this family: ClustalX and ClustalW. Clustal family tools are very fast and relatively reliable; they work in the same way as any other progressive alignment tool by either using PAM250 or BLOSUM62 with global DP for the pairwise alignment of all the provided sequences.

The pairwise scores are then used by Cluster to generate a neighbor joining tree; some early versions are used to follow the UPGMA method. The sequence alignment is done by following the leaves inward on the tree. Clustal is much more sensitive than other alignment tools as it considers the gap penalties and the short hydrophilic stretches as an indication of a random coil region so that the gap penalties are reduced for these stretches.

Clustal Omega: The most recent and best-performing webserver in the Clustal family. It can align around 200,000 sequences with just a single run and give results in a few hours. In terms of accuracy, it's very similar to other high-quality web servers, but on a large scale of sequencing, Clustal Omega is quite good in terms of both quality and the completion time of alignment. The very first step that this server follows is pairwise alignment by the k-tuple method. Afterward, the mBed method is used for clustering along with the K-means clustering method. Then it is followed by the building of a guide tree with the help of the UPGMA method. At last, the HHalign package aligns two hidden Markov model profiles and generates the alignment of multiple sequences.

4.7.5.2 *DIALign*

DIALign combines global and local pairwise alignments. DIALign has MSA, which is composed of segments of equal length that have statistical similarity, which is quite significant. DIALign is very similar to FASTA alignment.

4.7.5.3 *Tree-based consistency objective function for alignment evaluation (T-coffee)*

It is also a progressive multiple sequence alignment tool that uses ClustalW and LALIGN at the start to obtain a library of primary pairwise sequences that combine global pairwise sequences from ClustalW and local pairwise sequences from LALIGN. T-Coffee creates an extended library based on primary library triplets, where the third sequence aligns the other two sequences and creates a new pairwise alignment; duplicate pairs are removed completely. This extended library is actually a list of weighted residue pairs. From this extended library, the final multiple sequence alignment is generated by performing progressive alignment. 3D-coffee is a new and improved version of T-coffee. It uses a specialized server to create a 3D structure alignment, which shows a superposition, which is an area where two structures overlap. This superposition of two sequences is used to increase the weight of residues in the library (Notredame et al., 2000).

4.7.5.4 FAlign

FAlign uses a combination of both iterative and progressive algorithms to align multiple sequences. To use FAlign, users need to identify and define all the regions in all the sequences where a motif is present. At motif boundaries, all the sequences split, and the obtained segments are then aligned progressively. These segments are aligned, and they are properly assembled, which generates an alignment. BLOSUM62 is used in FAlign to create a score matrix that gives a sum-of-pair score. The alignment score is improved by the non-motif and by shifting the gaps iteratively and randomly.

4.7.6 Representation and structural inference

To generate multiple sequence alignments, we must first select more than two sequences from which to align and find similarity data. So after retrieving the sequences from FASTA, they are copied to the Clustal Omega web server box and run for the results. As it is very rapid, it will generate results within a few seconds. Using two examples, we will learn about the representation and structural inference of multiple sequence alignment.

Example 1: Sequences of different species are retrieved to use as input in clustal. The species were: panther histone, leopard mitochondria, tiger mitochondria, cat cloning vector, and synthetic cat (Fig. 4.10).

Because the sequences are from different species and have less similarity, there are no common matches between them. See any colon, semicolon, or star presence here that states the presence of matches or is very close to matches because the 5 sequences are very different from each other.

Example 2: Another five sequences of different organisms, but from the same class of species, Synthetic Cat, Cat Cloning Vector ThyA-Cat, Cat Expression Vector, and Cat Plasmid are the organisms (Fig. 4.11).

From the result, it is observed that there is alignment in the sequence. There are semicolons present that represent that they are much closer to the matches, and only 1 residue is different. Following that, there are colons, indicating that they are also close to the matches by the difference of two different residues.

4.8 Phylogenetics

Phylogenetic inference methods use heritable traits like DNA, amino acid sequences, or morphology to suggest links based on a trait evolution model. The outcome is a phylogeny (also known as a phylogenetic tree), a diagram demonstrating evolutionary links between species. Rooted and unrooted phylogenetic trees are the two forms of phylogenetic trees. A rooted tree diagram shows the tree's ancestry. An unrooted tree diagram (a network) has no assumptions about the ancestral line; therefore, it doesn't illustrate the origin or "root" of the species or the direction of anticipated evolutionary transitions.

Phylogenetic studies are used not just for their intended purpose of deducing phylogenetic patterns and taxa but also for the purpose of depicting connections among individual organisms or gene copies.

4.8.1 Molecular phylogenetics

The technique of inferring evolutionary relationships that develop as a result of molecular evolution and generating a phylogenetic tree is known as molecular phylogenetics. In molecular phylogenetics, genetic and hereditary molecular distinctions, especially DNA sequences, are analyzed to uncover an organism's evolutionary origins. A subset of molecular systematics, which uses molecular data in taxonomy and biogeography, is molecular phylogenetics. The fields of molecular phylogenetics and evolutionary biology are inextricably connected.

4.8.2 Phylogenetics trees

A phylogenetic tree, also known as an evolutionary tree or phylogeny, is a branching diagram that shows the evolutionary relationships between biological species or other entities based on physical or genetic similarities and differences. The lengths of the edges in some trees may be interpreted as estimations of the passage of time, and each node in a rooted phylogenetic tree represents the estimated most recent common ancestor. A taxonomic unit is a name that is assigned to each node in the tree. Internal nodes are sometimes called hypothetical taxonomic units because they are not easy to see.

The ancient idea of a hierarchical ladder connecting all forms of life gave rise to the modern “tree of life” metaphor. Edward Hitchcock's “paleontological chart” in his *Elementary Geology* book is one of the earliest examples of a “branching” phylogenetic tree because it shows the geological links between different species of plants and animals.

In his landmark essay *The Origin of Species*, published in 1859, Charles Darwin produced one of the earliest depictions and promoted the notion of an evolutionary “tree.” Even though they have been around for almost a century, tree diagrams are still widely used by evolutionary biologists to convey the idea that speciation happens through adaptive and semi-random branch splitting. Over time, species classification has grown increasingly fluid and less rigid.

4.8.3 Properties

A rooted phylogenetic tree is a directed tree that has one node at the root that corresponds to the (often accepted) most recent common ancestor of all the organisms at the tree's leaves. Despite the fact that it has no parents, the root node in the tree is the parent of all other nodes. As a result, the root is a node of degree 2, whereas the remainder of the internal nodes are all at least nodes of degree 3. (The term “degree” refers to the sum of all incoming and outgoing edges.)

Unrooted trees do not assume any ancestry and merely illustrate the relationship between the leaf nodes. If a tree's root is absent, it may always be created from an existing, rooted tree. On the other hand, determining an ancestor's lineage is necessary to infer a tree's root. Common ways to do this are to include an "outgroup" in the input data, force the root to be between the "outgroup" and the other species in the tree, or make extra assumptions about how fast each branch is changing.

Bifurcating and multifurcating branches are possible in both rooted and unrooted trees. Unrooted bifurcating trees have precisely three neighbors at each internal node, while rooted bifurcating trees (which form a binary tree) have precisely two descendants emanating from each inner node. On the other hand, a multifurcating, unrooted tree may have more than three neighbors. Trees can be labeled or unlabeled. Labeled trees have values assigned to the nodes, but an unlabeled tree, also known as a tree form, only depicts the structure of the tree (Kapli et al., 2020).

4.8.3.1 Phylogenetic trees and networks

Depending on the input data and technique used, computationally produced phylogenetic trees might be rooted or unrooted. Evolutionary processes like hybridization and horizontal gene transfer may be simulated by connecting rooted and unrooted phylogenetic trees to form rooted and unrooted phylogenetic networks. Dendrograms, phylograms, cladograms, and Dahlgren diagrams are only a few examples of the many tree representations that exist.

4.8.4 Building methods

Computational phylogenetics methods are used to generate phylogenetic trees from a large number of input sequences. Distance-matrix methods, such as neighbor-joining or UPGMA, which calculate genetic distance from multiple sequence alignments, are the easiest to construct. These methods aren't based on any sort of evolutionary theory. Simpler tree-building algorithms are used by several sequence alignment tools, including ClustalW. Maximum parsimony is another simple way to figure out phylogenetic trees, but it does require an implicit model of how evolution works.

4.8.5 Distance matrix method

To classify sequences, distance-matrix methods depend on a metric called "genetic distance." The percentage of mismatches at aligned sites is often used to calculate distance, with gaps either being disregarded or counted as mismatches. The goal of distance techniques is to create an all-to-all matrix that represents the distance between each pair of sequences in the sequence query set. A phylogenetic tree is made when sequences that are very similar to each other are put under the same inner node and the lengths of the branches are close to the distances between the sequences.

4.8.5.1 UPGMA

The UPGMA (Unweighted Pair Group Method with Arithmetic Mean) approach produces rooted trees and requires a constant-rate assumption, which necessitates an ultrametric tree with equal distances between root and branch tip. It is the oldest and simplest distance matrix method. It uses a sequential clustering algorithm and clusters the two closest species. The process is repeated until all species are grouped. The main advantage of this approach is that it is fast and is suitable for analyzing large data sets. There are several models available with many parameters in UPGMA that improve the distance estimation. However, this method lacks in some aspects, as a lot of information is lost, and the history of sites can only be investigated through character-based analyses.

4.8.5.2 Neighbor-joining

Neighbor-joining approaches extend general cluster analysis techniques to sequence analysis by using genetic distance as a grouping parameter. It is a method of clustering that works from the bottom up, and the term “neighbors” refers to taxa that are related to one another by a single node in an unrooted tree. The simple neighbor-joining method produces unrooted trees, but it does not assume that lineages evolve at the same pace. Up until the tree is resolved, the closest neighbors are sequentially linked by a new node. Even for many hundreds of sequences, the NJ technique is quick. However, the approach lacks accuracy because there is no attempt to correct for potential bias. NJ also does not examine all possible topologies.

4.8.5.3 Fitch Margoliash method

The Fitch-Margoliash method groups people based on their genetic distance by using a weighted least squares algorithm. To account for the higher inaccuracy in calculating distances between distantly related sequences, the tree-building strategy gives greater weight to tightly linked sequences. To prevent significant artifacts when computing relationships between closely and distantly related groups, the distances employed as the method’s input should be normalized. According to the distance linearity requirement, the anticipated value of the sum of two branch distances must be equal to the expected value of the two branch length sums. A substitution matrix, such as the one produced by the Jukes-Cantor model of DNA evolution, is used to make this correction. But the distance has to be changed when the branch rates start to change at different speeds.

Another variant of the strategy has been proven to increase the algorithm’s effectiveness and robustness, especially in cases involving relatively small distances. The least-squares criteria for distance measurement is more precise than neighbor-joining approaches but less time-efficient. At a higher processing cost, a further improvement can be added to take into account correlations between distances that come from a number of closely related sequences in the data set.

4.8.5.4 *Maximum parsimony*

Maximum parsimony (MP) is a method for deciding which phylogenetic tree best reflects the available sequencing data and needs the fewest overall evolutionary events. In this approach, sequences are aligned in order to build a tree that reduces the total number of mutations while simultaneously reducing the sum of all branch lengths. It aligns the sequences directly without making use of a distance matrix or an evolutionary model, and it entirely overlooks the notion that there may be numerous mutations. It was first designed to work with morphological characters. The fundamental idea behind MP is that it estimates the fewest possible replacements for any given topology. Among the benefits of this method are: (a) It is a simple method that doesn't make any assumptions and doesn't depend on any evolution model. However, if the substitution rates vary extensively between lineages, MP can also create incorrect topologies.

4.8.5.5 *Maximum likelihood*

The maximum likelihood approach assigns probabilities to fictitious evolutionary trees by using standard statistical methods for deriving probability distributions. The method calls for a substitution framework to calculate the probability of certain mutations; typically, a tree is judged to have a lower probability if it takes more mutations at inner nodes to account for the observed phylogeny. Maximum likelihood permits variable evolution rates across locales and lineages. As a result, this approach offers more statistical flexibility than maximum parsimony.

The "pruning" technique is frequently used to narrow the search space by more accurately predicting the likelihood of subtrees. The approach works backward from a node whose only descendants are leaves to the "bottom" node, estimating the probability for each site in nested sets in a linear manner. The procedure will only result in the production of rooted trees if the replacement model is irreversible, which isn't usually the case in biology.

This approach offers the most compelling theoretical justification for its use. Experiments using sequence simulation have demonstrated that this approach is superior to all others in terms of effectiveness in the majority of scenarios. However, this strategy relies heavily on the use of computers. Because there are so many different trees to consider, it is almost never possible to examine them all. A preliminary investigation of potential tree locations is carried out. The mathematical certainty of generating the tree with the highest probability has been lost.

4.8.6 Bayesian inference

In a manner similar to maximum likelihood methods, phylogenetic trees may be constructed using Bayesian inference. Bayesian approaches estimate a prior probability distribution of the potential trees. This can be as simple as the probability of any one tree out of all the possible trees that can be made from the data, or it can be a more complicated estimate based on the idea that divergence events like speciation

are random. The prior distribution that is used in Bayesian inference phylogenetics is seen differently by different users.

Examples of Bayesian phylogenetics choices include randomly swapping the descendant subtrees of a random internal node between two interrelated trees and performing a circular permutation of the leaf nodes of a postulated tree at every phase. The use of Bayesian approaches in phylogenetics has been met with much debate as a result of a lack of transparency in previously published research about the decision-making process for the prior distribution, acceptance criterion, and move set.

Bayesian procedures use the posterior distribution to reconstruct a tree that depicts the most likely clades, whereas likelihood approaches locate the tree with the highest probability given the data. It is possible, however, for estimates of clade posterior probability to be quite inaccurate, especially for clades that aren't extremely probable. This has led to the introduction of several strategies for estimating the posterior probability.

References

- Altschul, S.F., Gish, W., Miller, W., Myers, E.W., Lipman, D.J., 1990. Basic local alignment search tool. *J. Mol. Biol.* 215, 403–410. [https://doi.org/10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2).
- Edgar, R., Batzoglou, S., 2006. Multiple sequence alignment. *Curr. Opin. Struct. Biol.* 16.
- Gotoh, O., 1999. Multiple sequence alignment: Algorithms and applications. *Adv. Biophys.* 36.
- Henikoff, S., Henikoff, J.G., 1992. Amino acid substitution matrices from protein blocks. *Proc. Natl. Acad. Sci. U. S. A.* 89, 10915–10919. <https://doi.org/10.1073/PNAS.89.22.10915>.
- Kapli, P., Yang, Z., Telford, M.J., 2020. Phylogenetic tree building in the genomic age. *Nat. Rev. Genet.* 21 (7), 428–444. <https://doi.org/10.1038/s41576-020-0233-0>.
- Mullan, L., 2006. Pairwise sequence alignment - it's all about us! *Briefings Bioinf.* 7, 113–115. <https://doi.org/10.1093/BIB/BBK008>.
- Naghizadeh, M., Babaei, S., Behkmal, B., Hatami, M., 2021. Divide and conquer approach to long genomic sequence alignment. In: *ICCKE 2021 - 11th International Conference on Computer Engineering and Knowledge*, pp. 399–405. <https://doi.org/10.1109/ICCKE54056.2021.9721501>.
- Needleman, S.B., Wunsch, C.D., 1970. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *J. Mol. Biol.* 48, 443–453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4).
- Notredame, C., Higgins, D.G., Heringa, J., 2000. T-coffee: A novel method for fast and accurate multiple sequence alignment. *J. Mol. Biol.* 302, 205–217. <https://doi.org/10.1006/jmbi.2000.4042>.
- Pearson, W.R., 2013. Selecting the right similarity-scoring matrix. *Curr. Protoc. Bioinformatics* 43, 3.5.1–3.5.9. <https://doi.org/10.1002/0471250953.BI0305S43>.
- Rice, P., Longden, L., Bleasby, A., 2000. EMBOSS: The European molecular biology open software suite. *Trends Genet.* 16, 276–277. [https://doi.org/10.1016/S0168-9525\(00\)02024-2](https://doi.org/10.1016/S0168-9525(00)02024-2).

- Saloom, R.H., Khafaji, H.K., Tobia, R.H., 2022. A survey for the methods of detection and classification of genetic mutations. *Indones. J. Electr. Eng. Comput. Sci.* 28, 1796–1816. <https://doi.org/10.11591/IJEECS.V28.I3.PP1796-1816>.
- Smite, T.F., Waterman, M.S., 1981. Identification of common molecular subsequences. *J. Mol. Biol.* 147, 195–197.
- Sun, J., Chen, K., Hao, Z., 2018. Pairwise alignment for very long nucleic acid sequences. *Biochem. Biophys. Res. Commun.* 502, 313–317. <https://doi.org/10.1016/J.BBRC.2018.05.134>.
- Taylor, W., 1986. Identification of protein sequence homology by consensus template alignment. *J. Mol. Biol.* 188.